

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №5
по курсу «Алгоритмы и структуры данных»
Тема: Деревья. Пирамида, пирамидальная сортировка. Очередь с
приоритетами
Вариант 7 (Продвинутый уровень)

Выполнила:
Кулакова Елизавета Алексеевна
К3260

Проверил:
Харлуниин А. А.

Санкт-Петербург
2025 г.

Содержание отчета

Содержание отчета.....	2
Задачи по варианту (базовый уровень).....	3
Задача №2. Высота дерева.....	3
Задача №3. Обработка сетевых пакетов.....	9
Задачи на продвинутый уровень.....	22
Задача №1. Куча ли?.....	22
Задача №5. Планировщик задач.....	27
Вывод.....	35

Задачи по варианту (базовый уровень)

Задача №2. Высота дерева

В этой задаче ваша цель - привыкнуть к деревьям. Вам нужно будет прочитать описание дерева из входных данных, реализовать структуру данных, сохранить дерево и вычислить его высоту.

- Вам дается корневое дерево. Ваша задача - вычислить и вывести его высоту. Напомним, что высота (корневого) дерева - это максимальная глубина узла или максимальное расстояние от листа до корня. Вам дано произвольное дерево, не обязательно бинарное дерево.
- Формат ввода или входного файла (input.txt).** Первая строка содержит число узлов n ($1 \leq n \leq 10^5$). Вторая строка содержит n целых чисел от -1 до $n-1$ – указание на родительский узел. Если i -ое значение равно -1 , значит, что узел i - корневой, иначе это число является обозначением индекса родительского узла этого i -го узла ($0 \leq i \leq n-1$). **Индексы считать с 0.** Гарантируется, что дан только один корневой узел, и что входные данные представляют дерево.
- Формат вывода или выходного файла (output.txt).** Выведите целое число – высоту данного дерева.
- Ограничение по времени. 3 сек.
- Ограничение по памяти. 512 мб.
- Пример 1:

input.txt	output.txt
5	3
4 -1 4 1 1	

- Объяснение примера.** Данный входной файл задает 5 узлов дерева с числами от 0 до 4. Узел под индексом 0 является дочерним узлом узла с индексом 4. Узел под индексом 1 - корневой узел. Узел с индексом 2 - тоже дочерний узел четвертого узла, а узлы с индексами 3 и 4 - дочерние узлы первого (корневого) узла. Можно записать данные узлы с соответствующими индексами, чтобы увидеть наглядно:

0	1	2	3	4
4	-1	4	1	1

Давайте построим это дерево: Узел «1» – корневой, у него двое дочерних узла: «3» и «4», и у узла «4» – тоже два дочерних: «0» и «2».

Таким образом, высота этого дерева равна 3, т.к. количество вершин на пути от корня 1 к листу 2 равно 3.

- Пример 2:

input.txt	output.txt
5	4
-1 0 4 0 3	

Листинг кода

```
from collections import deque
from generator import *
from measurments import *

def build_tree(parent):
    n = len(parent)
    children = [[] for _ in range(n)]
```

```

    root = -1
    for i in range(n):
        if parent[i] == -1:
            root = i
        else:
            children[parent[i]].append(i)
    return root, children

@measure_performance
def get_tree_height(tree):
    if tree is None:
        return 0

    root, children = build_tree(tree)
    if root == -1:
        return 0

    q = deque([root])
    height = 0

    while q:
        level_size = len(q)
        for _ in range(level_size):
            v = q.popleft()
            for child in children[v]:
                q.append(child)
            height += 1

    return height

if __name__ == "__main__":
    tree_generator(10**5)
    with open('input.txt') as f:
        n = int(f.readline())
        array = list(map(int, f.readline().split()))
    h = get_tree_height(array)
    with open('output.txt', 'w') as f:
        f.write(str(h))
    print(h)

```

Листинг кода функции для формирования массива

```

def tree_generator(n=None):
    if n is None:
        n = random.randint(1, 10**5)
    tree = random.choices(range(0, n), k=n)
    root_index = random.randint(0, n-1)
    tree[root_index] = -1

    with open('input.txt', 'w') as f:
        f.write(f'{n}\n')
        f.write(" ".join(map(str, tree)))

    return None

```

Текстовое объяснение решения.

Построение структуры дерева

Сначала из массива `parent` строится массив списков `children`, где `children[v]` хранит всех детей вершины `v`, а также определяется корень: это вершина, для которой `parent[i] == -1`.

Такое представление естественно для произвольного дерева (не обязательно бинарного) и позволяет быстро получать всех детей любой вершины.

Алгоритм вычисления высоты

Высота дерева считается итеративно с помощью обхода в ширину (BFS) по уровням от корня.

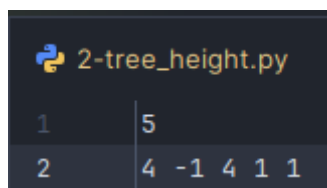
В очередь сначала кладётся один корень; затем в цикле `while` обрабатывается текущий уровень: для всех вершин этого уровня в очередь добавляются их дети, после чего счётчик `height` увеличивается на 1. Когда очередь пуста, `height` равен числу уровней дерева, то есть количеству вершин на самом длинном пути от корня до листа.

Работа с вводом и генерацией тестов

Функция `get_tree_height` принимает массив родителей, внутри строит структуру дерева и возвращает одно целое число - высоту. В `main`-части программа читает `n` и массив `parent` из файла `input.txt`, вызывает подсчёт высоты и записывает результат в `output.txt`.

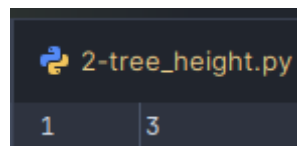
Дополнительно есть генератор `tree_generator`, который формирует случайный массив родителей для `n` узлов: выбирается случайный корневой индекс и ему присваивается значение `-1`, а остальным вершинам - случайные родители; результат сразу сохраняется в файл `input.txt`, что позволяет тестировать алгоритм на максимальных размерах входа.

Результат работы кода:



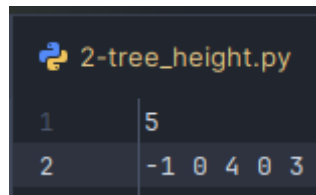
1	5
2	4 -1 4 1 1

Рисунок 1 – Файл `input.txt` с входными данными из примера 1



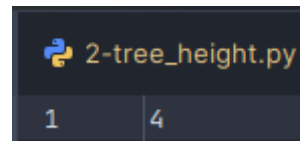
1	3
---	---

Рисунок 2 – Результат работы функции, пример из задания 1



1	5
2	-1 0 4 0 3

Рисунок 3 – Файл input.txt с входными данными из примера 2



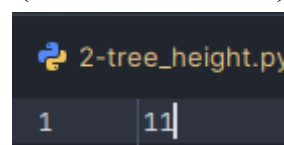
1	4
2	

Рисунок 4 – Результат работы функции, пример из задания 2



1	54247
2	11790 10914 43840 51553 22782 13237 46020 19934 2438 22594 19658 16299 26239 18606 48218 36349 37732 31865 44326 13733
	< 42974 38981 4180 27017 23815 43758 40458 28657 2020 15664 20578 23255 2390 777 16089 27672 18898 15519 32231 24725
	< 18786 16323 27061 46365 42640 30556 49163 43284 37942 8060 16180 30258 21623 5930 35672 30134 39720 32556 53726 24695
	< 27459 4139 23539 23520 17751 25025 11289 43759 10090 46381 43578 52255 33177 52802 31262 40610 267 23325 44800 39384
	< 12087 52676 38195 7403 44320 6532 16554 20130 14407 48642 14917 18695 48691 8685 34649 3483 12767 52023 24302 23909
	< 2259 15659 45280 51867 14457 13896 39602 46580 18862 24273 53520 12291 44562 42976 46723 3554 12700 11616 51249 19645
	< 14911 37000 47398 23404 4239 48962 29793 48036 42598 18279 11487 33905 34867 11818 35012 38214 9844 49223 45146 31980
	< 45141 27161 16693 17159 53474 9656 32289 7679 41784 13136 38603 22153 44187 33043 31783 17337 2926 39664 18918 26304
	< 20419 5628 23977 45023 35063 10942 43578 50286 47085 43931 42246 32512 30879 32512 42942 43584 8494 6373 25477 41707
	< 5587 7530 29992 44850 53429 46737 30487 52044 1218 15143 36432 43454 49877 24016 10157 29458 31161 52791 10707 18486
	< 52815 6475 20994 682 39755 44907 21407 32350 9242 41707 45725 23188 21048 51777 9110 49321 44533 48137 5785 42433
	< 43405 7 41702 39502 23173 53250 33472 910 47270 11598 5548 15721 16849 50191 21770 48544 19640 14542 40469 12055 9966
	< 52101 2 12918 48177 49182 21859 33682 13060 4581 52540 39883 18020 21062 34577 32056 51611 38197 1393 27745 46128 8191
	< 4015 33691 5515 17754 5720 35964 20716 12605 53698 26114 8793 19962 18583 20267 5628 12490 31428 36398 45644 23722
	< 51592 7976 32652 45345 29625 28658 18593 35734 26400 23001 29311 44901 39086 47352 40983 36271 20606 43908 18354 2642
	< 24341 35208 5410 33168 15069 8548 27923 7186 7111 43650 23146 52328 10764 34460 10661 23869 53844 16697 28682 54008
	< 10031 15227 22285 38386 30729 31983 34146 43735 23677 32457 23234 18155 21547 1258 4252 40126 33629 37459 52306 31997
	< 42903 28171 37179 39600 49520 29646 53804 33507 46453 22443 4787 49472 13988 50097 1740 47961 951 18299 11221 27087
	< 1323 40474 27827 41228 46173 16344 53570 4122 9423 1009 21215 15394 29389 32960 14917 36095 27774 50975 18860 27763
	< 5331 13651 53278 36300 13501 11565 27 29296 29903 31968 3767 8328 17452 46870 31505 7790 26508 40523 32100 37790 53929
	< 51627 44441 30702 42132 52914 44277 23642 6034 12071 32793 29766 9941 30035 12882 15868 36411 9139 35441 42108 32344
	< 7178 4937 9539 52944 26152 51406 28079 46176 53580 52955 40528 49621 41185 2487 16749 25161 20548 1559 19288 29606
	< 0017 74445 27425 18570 2770 22540 0150 10100 52000 15704 8500 11250 3107 74427 20776 7071 12454 35370 14427 40547

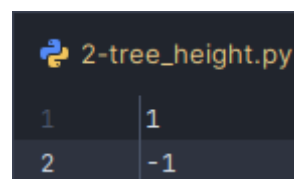
Рисунок 5 – Файл input.txt со сгенерированными входными данными (54247 значений)



1	11
2	

Рисунок 6 – Результат работы функции, сгенерированные входные данные

Результат работы кода на максимальных и минимальных значениях:



1	1
2	-1

Рисунок 7 – Файл input.txt с вводными данными – минимальными значениями

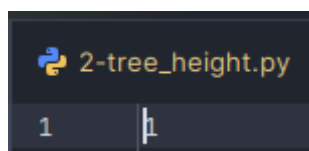


Рисунок 8 – Файл output.txt после выполнения функции с минимальными значениями

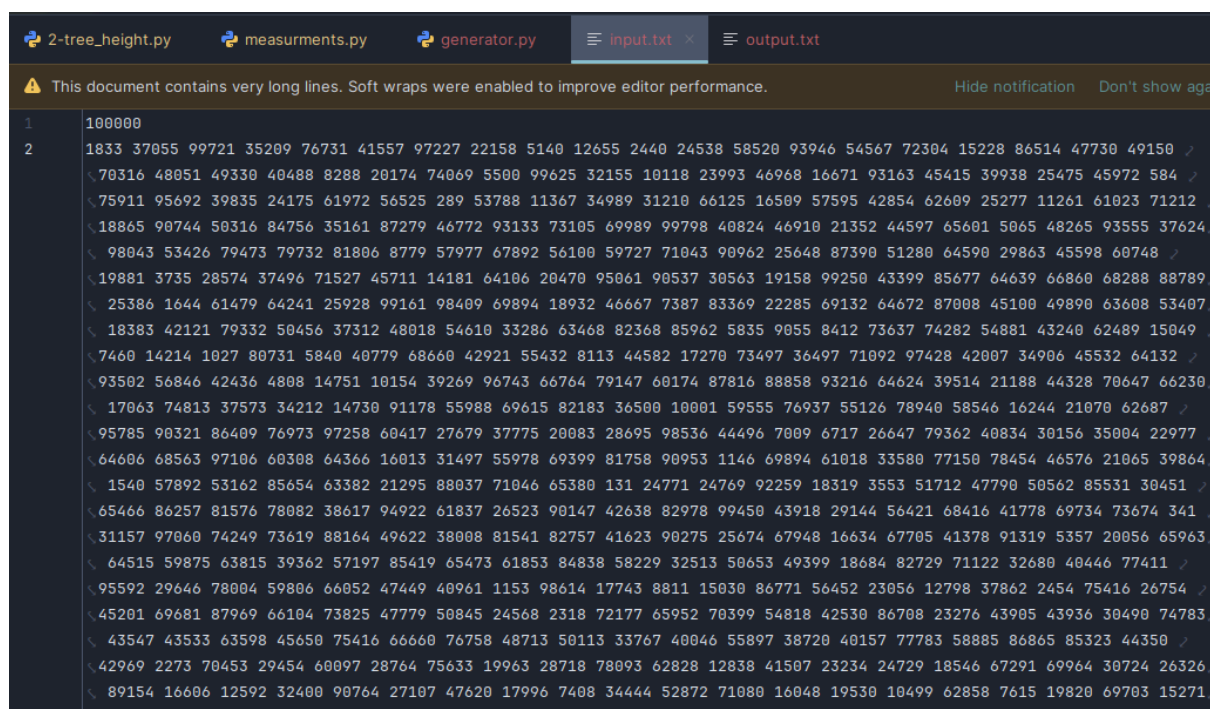


Рисунок 9 – Файл input.txt с вводными данными – максимальными значениями

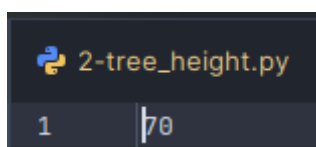


Рисунок 10 – Файл output.txt после выполнения функции с максимальными значениями

Таблица затрат памяти и времени по заданию 2:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0000 сек	0.92 KB
Сгенерированный пример	0.1104 сек	6.02 MB
Пример 1	0.0000 сек	1.02 KB
Пример 2	0.0000 сек	1.05 KB

Верхняя граница диапазона значений входных данных из текста задачи	0.1530 сек	11.09 МВ
--	------------	----------

Вывод по задаче: решение корректно использует массив индексов как удобное представление вершин, за один проход строит структуру дерева и затем за один обход по уровням находит высоту. Вся обработка выполняется за линейное время $O(n)$ и с линейной памятью, что укладывается в ограничения по времени и памяти и работает для деревьев произвольной формы и глубины.

Задача №3. Обработка сетевых пакетов

В этой задаче вы реализуете программу для моделирования обработки сетевых пакетов.

- Вам дается серия входящих сетевых пакетов, и ваша задача - смоделировать их обработку. Пакеты приходят в определенном порядке. Для каждого номера пакета i вы знаете время, когда пакет прибыл A_i и время, необходимое процессору для его обработки P_i (в миллисекундах). Есть только один процессор, и он обрабатывает входящие пакеты в порядке их поступления. Если процессор начал обрабатывать какой-либо пакет, он не прерывается и не останавливается, пока не завершит обработку этого пакета, а обработка пакета i занимает ровно P_i миллисекунд.

Компьютер, обрабатывающий пакеты, имеет сетевой буфер фиксированного размера S . Когда пакеты приходят, они сохраняются в буфере перед обработкой. Однако, если буфер заполнен, когда приходит пакет (есть S пакетов, которые прибыли до этого пакета, и компьютер не завершил обработку ни одного из них), он отбрасывается и не обрабатывается вообще. Если несколько пакетов поступают одновременно, они сначала все сохраняются в буфере (из-за этого некоторые из них могут быть отброшены - те, которые описаны позже во входных данных). Компьютер обрабатывает пакеты в порядке их поступления и начинает обработку следующего доступного пакета из буфера, как только заканчивает обработку предыдущего. Если в какой-то момент компьютер не занят и в буфере нет пакетов, компьютер просто ожидает прибытия следующего пакета. Обратите внимание, что пакет покидает буфер и освобождает пространство в буфере, как только компьютер заканчивает его обработку.

- Формат ввода или входного файла (input.txt).** Первая строка содержит размер S буфера ($1 \leq S \leq 10^5$) и количество n ($1 \leq n \leq 10^5$) входящих сетевых пакетов. Каждая из следующих n строк содержит два числа, i -ая строка содержит время прибытия пакета A_i ($0 \leq A_i \leq 10^6$) и время его обработки P_i ($0 \leq P_i \leq 10^3$) в миллисекундах. Гарантируется, что последовательность времени прибытия входящих пакетов – неубывающая, однако, она может содержать одинаковые значения времени прибытия нескольких пакетов, в этом случае рассматривается пакет, записанный в входном файле раньше остальных, как прибывший ранее. ($A_i \leq A_{i+1}$ для $1 \leq i \leq n - 1$.)
- Формат вывода или выходного файла (output.txt).** Для каждого пакета напечатайте время (в миллисекундах), когда процессор начал его обрабатывать; или -1, если пакет был отброшен. Вывести ответ нужно в том же порядке, как как пакеты были описаны во входном файле.
- Ограничение по времени. 10 сек.
- Ограничение по памяти. 512 мб.
- Пример 1:

input.txt	output.txt
1 0	

Если нет пакетов, ничего выводить не нужно.

- Пример 2:

input.txt	output.txt
1 1	0
0 0	

Единственный пакет поступил в момент времени 0, и компьютер обработал его сразу.

- Пример 3:

input.txt	output.txt
1 2	0
0 1	-1
0 1	

Первый пакет поступил в момент времени 0, второй пакет также, но был отброшен, так как сетевой буфер имеет размер 1 и он был полон (т.к. место занят первый пакет). Первый пакет начал обрабатываться в момент времени 0, а второй не обрабатывался.

- Пример 4:

input.txt	output.txt
1 2	0
0 1	1
1 1	

Первый пакет поступил в момент времени 0, компьютер сразу начал его обрабатывать и закончил в момент времени 1. Второй пакет поступил во время 1, и компьютер так же начал его обрабатывать.

- Что делать. Для решения этой задачи вы можете использовать массив или очередь (точнее, дек, чтобы у вас был доступ к последнему элементу).

Одно из возможных решений - сохранить в списке или очереди время, когда компьютер завершит обработку пакетов (*finish_time*), которые в настоящее время хранятся в сетевом буфере, в порядке возрастания. Когда прибывает новый пакет, вам сначала нужно удалить в начале *finish_time* все пакеты, которые уже обработаны к моменту прибытия нового пакета. Затем вы добавляете время окончания для нового пакета в *finish_time*. Если буфер заполнен (в *очереди finish_time* уже *S* элементов), пакет отбрасывается. В противном случае время завершения его обработки добавляется к *finish_time*.

Если при поступлении нового пакета очередь *finish_time* пуста, компьютер начнет обработку нового пакета немедленно, как только он поступит.

В противном случае компьютер начнет обработку нового пакета, как только он закончит обработку последнего из пакетов, находящихся в настоящее время в `finish_time` (здесь вам нужно получить доступ к последнему элементу `finish_time`, чтобы определить, когда компьютер начнет обрабатывать новый пакет). Вам также нужно будет вычислить время окончания обработки, добавив P_i к времени начала обработки и поместив его в конец `finish_time`.

- Еще примеры.

№	input.txt	output.txt	№	input.txt	output.txt	№	input.txt	output.txt
5	1 1 0 1	0	10	1 2 0 1 0 1	0 -1	14	3 6 0 2 1 2 2 2 3 2 4 2 5 2	0 2 4 6 8 -1
6	1 1 1 0	1	11	1 2 0 1 1 1	0 1			
7	1 2 0 0 0 0	0 0	12	1 2 0 1 2 1	0 2			
8	1 2 0 0 0 1	0 0	13	2 3 0 1 3 1 10 1	0 3 10			
9	1 2 0 1 0 0	0 -1						

Листинг кода.

```
from collections import deque
from measurments import *
from generator import *

@measure_performance
def pack_preproc(arr, buffer_capacity):
    if len(arr) <= 1:
        if len(arr) == 0:
            return []
        else:
            return [arr[0][0]]
    cur_time = 0
    deq = deque()
    process_start = [None] * len(arr)

    for i in range(len(arr)):
        a = arr[i]
        if len(deq) < buffer_capacity:
            deq.append((a, i))
        else:
            if cur_time < deq[0][0][0]:
                cur_time = deq[0][0][0]
            process_start[deq[0][1]] = cur_time
            cur_time += deq[0][0][1]
            deq.popleft()
            if cur_time > a[0]:
                process_start[i] = -1
            else:
                deq.append((a, i))

    while deq:
```

```

        if cur_time < deq[0][0][0]:
            cur_time = deq[0][0][0]
        process_start[deq[0][1]] = cur_time
        cur_time += deq[0][0][1]
        deq.popleft()

    return process_start

if __name__ == '__main__':
    packages_generator(10**5, 10)
    with open('input.txt') as f:
        bc, n = tuple(map(int, f.readline().split()))
        packs = []
        for _ in range(n):
            num, time = tuple(map(int, f.readline().split()))
            packs.append((num, time))

    res = pack_preproc(packs, bc)
    with open('output.txt', 'w') as f:
        f.write("\n".join(map(str, res)))

```

Листинг кода дополнительной функции.

```

def packages_generator(n=None, s=None):
    if n is None:
        n = random.randint(1, 10**5)
    if s is None:
        s = random.randint(1, 10**5)

    at = sorted(random.choices(range(0, 10**6 + 1), k=n))
    pt = random.choices(range(0, 10**3 + 1), k=n)

    with open('input.txt', 'w') as f:
        f.write(f'{s} {n}')
        for a, p in zip(at, pt):
            f.write(f'\n{a} {p}')
    return None

```

Текстовое объяснение решения.

Алгоритм моделирует работу сетевого буфера и процессора по времени. На вход подаётся список пакетов в порядке поступления, каждый с моментом прихода и временем обработки, и фиксированный размер буфера. Под каждый пакет создаётся ячейка в массиве ответов, куда позже записывается либо время начала обработки, либо -1, если пакет был отброшен. Для моделирования буфера используется очередь, в которой хранятся пары «данные пакета + его исходный индекс». Переменная текущего времени отражает, когда процессор освободится.

При приходе очередного пакета сначала проверяется состояние буфера. Если в буфере ещё есть место, пакет просто добавляется в конец очереди. Если буфер уже заполнен, значит, нужно попытаться продвинуть

обработку: из начала очереди берётся самый ранний пакет, при необходимости текущее время «догоняется» до момента его прихода, затем этому пакету назначается время старта, текущее время увеличивается на его длительность обработки, и пакет удаляется из очереди, освобождая место. После этого рассматривается новый пакет: если он «пришёл в прошлое» относительно уже продвинутого текущего времени, это означает, что к моменту его прихода буфер всё равно был заполнен, и такой пакет помечается как отброшенный (в массив ответов записывается -1). Если же его время прихода не конфликтует с текущим временем, он добавляется в очередь ожидания. Когда все входные пакеты рассмотрены, очередь может ещё содержать необработанные элементы, и тогда алгоритм последовательно завершает обработку оставшихся пакетов: для каждого при необходимости подтягивает текущее время до его момента прихода, записывает в ответы время начала обработки и добавляет длительность к текущему времени. В итоге для каждого пакета, в исходном порядке, известно либо точное время начала его обработки, либо факт отбрасывания; такая симуляция корректно учитывает порядок прихода, ограниченный размер буфера и непрерывную обработку без прерываний.

Функция `packages_generator` служит для автоматической подготовки тестового файла в том же формате, в каком его будет читать основная программа. Если параметры n (количество пакетов) и s (размер буфера) явно не переданы, они случайно выбираются в допустимых границах от 1 до 10^5 , чтобы можно было тестировать алгоритм на разных размерах входа. Далее формируются два набора случайных чисел: массив моментов прихода at и массив длительностей обработки pt . Времена прихода сначала выбираются случайно в диапазоне от 0 до 10^6 , а затем сортируются, чтобы удовлетворить требованию неубывающей последовательности времени поступления пакетов; времена обработки выбираются независимо в допустимом диапазоне от 0 до 10^3 . Затем функция открывает файл `input.txt` на запись и в первой строке записывает два числа: размер буфера s и количество пакетов n . В следующих строках она построчно записывает пары значений «время прихода – время обработки» для каждого пакета, строго в том порядке, в котором алгоритм потом будет их читать и обрабатывать. Таким образом, генератор позволяет быстро получать множество корректных случайных тестов, полностью соответствующих формату задачи, и проверять, как работает алгоритм обработки пакетов на больших и разнообразных входных данных.

Результат работы кода на примерах из задания:

```

3-package
1 0

```

Рисунок 11 – Файл input.txt с входными данными из примера 1

```

3-packages_p
1 0

```

Рисунок 12 – Результат работы кода для данных из примера 1, записанный в output.txt

```

3-packages
1 1
2 0

```

Рисунок 13 – Файл input.txt с входными данными из примера 2

```

3-packa
1 0

```

Рисунок 14 – Результат работы кода для данных из примера 2, записанный в output.txt

```

3-packages
1 2
2 0 1
3 0 1

```

Рисунок 15 – Файл input.txt с входными данными из примера 3

```

3-packag
1 0
2 -1

```

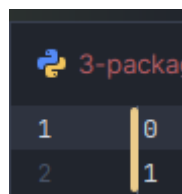
Рисунок 16 – Результат работы кода для данных из примера 3, записанный в output.txt

```

3-packages
1 2
2 0 1
3 1 1

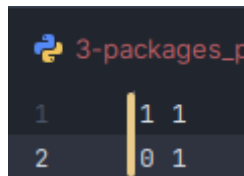
```

Рисунок 17 – Файл input.txt с входными данными из примера 4



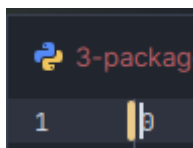
1	0
2	1

Рисунок 18 – Результат работы кода для данных из примера 4, записанный в output.txt



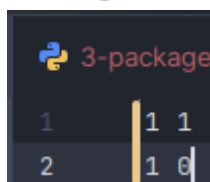
1	1	1
2	0	1

Рисунок 19 – Файл input.txt с входными данными из примера 5



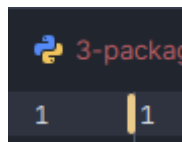
1	0
---	---

Рисунок 20 – Результат работы кода для данных из примера 5, записанный в output.txt



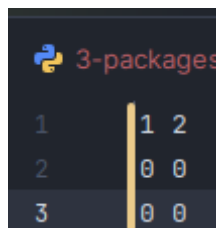
1	1	1
2	1	0

Рисунок 21 – Файл input.txt с входными данными из примера 6



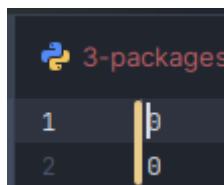
1	1
---	---

Рисунок 22 – Результат работы кода для данных из примера 6, записанный в output.txt



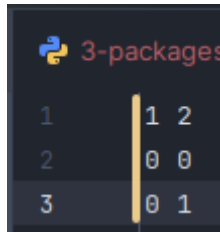
1	1	2
2	0	0
3	0	0

Рисунок 23 – Файл input.txt с входными данными из примера 7



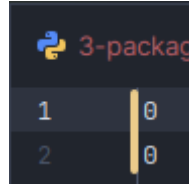
1	0
2	0

Рисунок 24 – Результат работы кода для данных из примера 7, записанный в output.txt



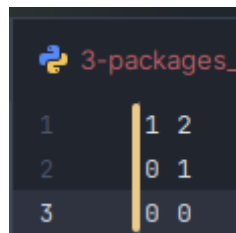
1	1	2
2	0	0
3	0	1

Рисунок 25 – Файл input.txt с входными данными из примера 8



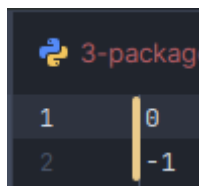
1	0
2	0

Рисунок 26 – Результат работы кода для данных из примера 8, записанный в output.txt



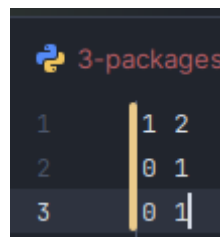
1	1	2
2	0	1
3	0	0

Рисунок 27 – Файл input.txt с входными данными из примера 9



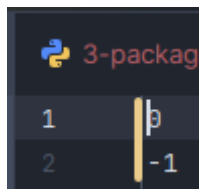
1	0
2	-1

Рисунок 28 – Результат работы кода для данных из примера 9, записанный в output.txt



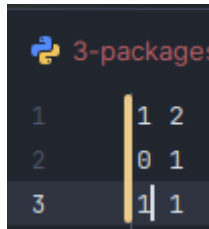
1	1	2
2	0	1
3	0	1

Рисунок 29 – Файл input.txt с входными данными из примера 10



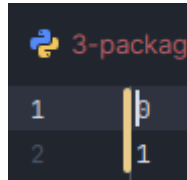
1	0
2	-1

Рисунок 30 – Результат работы кода для данных из примера 10, записанный в output.txt



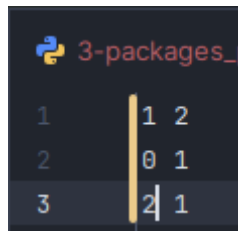
1	1 2
2	0 1
3	1 1

Рисунок 31 – Файл input.txt с входными данными из примера 11



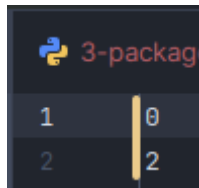
1	0
2	1

Рисунок 32 – Результат работы кода для данных из примера 11, записанный в output.txt



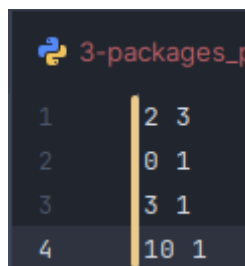
1	1 2
2	0 1
3	2 1

Рисунок 33 – Файл input.txt с входными данными из примера 12



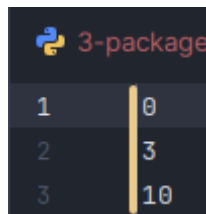
1	0
2	2

Рисунок 34 – Результат работы кода для данных из примера 12, записанный в output.txt



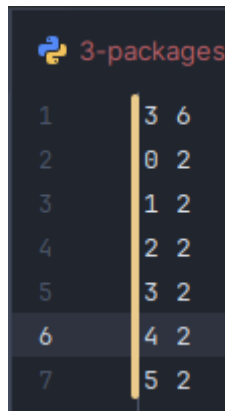
1	2 3
2	0 1
3	3 1
4	10 1

Рисунок 35 – Файл input.txt с входными данными из примера 13



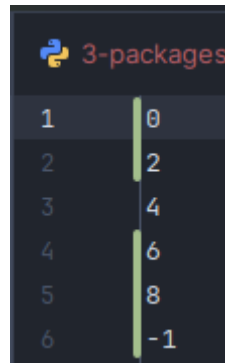
1	0
2	3
3	10

Рисунок 36 – Результат работы кода для данных из примера 13, записанный в output.txt



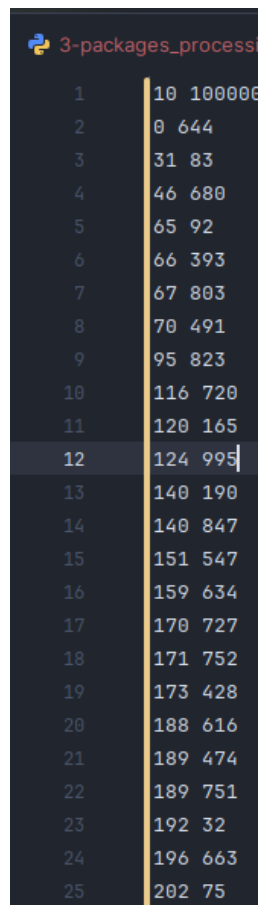
1	3	6
2	0	2
3	1	2
4	2	2
5	3	2
6	4	2
7	5	2

Рисунок 37 – Файл input.txt с входными данными из примера 14



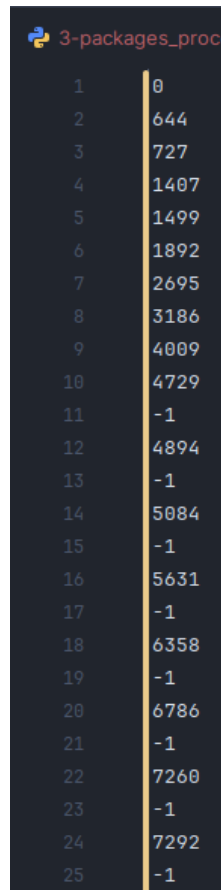
1	0
2	2
3	4
4	6
5	8
6	-1

Рисунок 38 – Результат работы кода для данных из примера 14, записанный в output.txt



1	10	100000
2	0	644
3	31	83
4	46	680
5	65	92
6	66	393
7	67	803
8	70	491
9	95	823
10	116	720
11	120	165
12	124	995
13	140	190
14	140	847
15	151	547
16	159	634
17	170	727
18	171	752
19	173	428
20	188	616
21	189	474
22	189	751
23	192	32
24	196	663
25	202	75

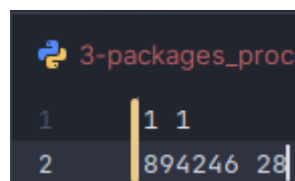
Рисунок 39 – Файл input.txt со сгенерированными входными данными
(буфер размера 10, 100000 пакетов)



1	0
2	644
3	727
4	1407
5	1499
6	1892
7	2695
8	3186
9	4009
10	4729
11	-1
12	4894
13	-1
14	5084
15	-1
16	5631
17	-1
18	6358
19	-1
20	6786
21	-1
22	7260
23	-1
24	7292
25	-1

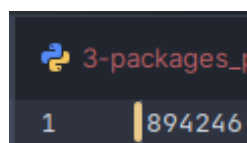
Рисунок 40 – Результат работы кода для сгенерированных данных,
записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:



1	1	1
2	894246	28

Рисунок 41 – Минимальное вводимое значение в input.txt



1	894246
---	--------

Рисунок 42 – Результат работы кода для значения s, n = 1, записанный в
output.txt

```

3-packages_processing.py
1 100000 100000
2 0 234
3 8 450
4 16 768
5 24 64
6 48 258
7 78 850
8 86 550
9 87 835
10 95 743
11 95 423
12 96 612
13 100 135
14 123 74
15 136 296
16 146 388
17 150 700
18 152 986
19 163 427
20 172 14
21 173 43
22 195 104
23 218 933
24 227 980
25 228 162

```

Рисунок 43 – Максимальное вводимое значение в input.txt

```

3-packages_proce
1 0
2 234
3 684
4 1452
5 1516
6 1774
7 2624
8 3174
9 4009
10 4752
11 5175
12 5787
13 5922
14 5996
15 6292
16 6680
17 7380
18 8366
19 8793
20 8807
21 8850
22 8954
23 9887
24 10867
25 11029

```

Рисунок 44 – Результат работы кода для значения s, n = 100000, записанный в output.txt

Таблица затрат памяти и времени по заданию 3:

	Время выполнения	Затраты памяти
Нижняя граница	0.0000 сек	0.07 KB

диапазона значений входных данных из текста задачи		
Пример 1	0.0000 сек	0.06 KB
Пример 2	0.0000 сек	0.07 KB
Пример 3	0.0000 сек	0.91 KB
Пример 4	0.0000 сек	0.91 KB
Пример 5	0.0000 сек	0.07 KB
Пример 6	0.0000 сек	0.07 KB
Пример 7	0.0000 сек	0.91 KB
Пример 8	0.0000 сек	0.91 KB
Пример 9	0.0000 сек	0.91 KB
Пример 10	0.0000 сек	0.91 KB
Пример 11	0.0000 сек	0.91 KB
Пример 12	0.0000 сек	0.91 KB
Пример 13	0.0000 сек	0.91 KB
Пример 14	0.0000 сек	0.94 KB
Собственный пример	0.0512 сек	2.29 MB
Верхняя граница диапазона значений входных данных из текста задачи	0.1500 сек	9.94 MB

Вывод по задаче:

Алгоритм реализует точную симуляцию системы с одним процессором и буфером ограниченного размера: пакеты обрабатываются по порядку прихода, без прерываний, отбрасываются только тогда, когда буфер полностью занят к моменту их прихода, и для каждого пакета корректно вычисляется время начала обработки или факт отбрасывания. Такое решение линейно по числу пакетов и укладывается в заданные ограничения по времени и памяти.

Задачи на продвинутый уровень

Задача №1. Куча ли?

Структуру данных «куча», или, более конкретно, «неубывающая пирамида», можно реализовать на основе массива.

Для этого должно выполняться основное свойство неубывающей пирамиды, которое заключается в том, что для каждого $1 \leq i \leq n$ выполняются условия:

1. если $2i \leq n$, то $a_i \leq a_{2i}$,
2. если $2i + 1 \leq n$, то $a_i \leq a_{2i+1}$.

Дан массив целых чисел. Определите, является ли он неубывающей пирамидой.

- **Формат входного файла (input.txt).** Первая строка входного файла содержит целое число n ($1 \leq n \leq 10^6$). Вторая строка содержит n целых чисел, по модулю не превосходящих $2 \cdot 10^9$.
- **Формат выходного файла (output.txt).** Выведите «YES», если массив является неубывающей пирамидой, и «NO» в противном случае.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.
- Примеры:

№	input.txt	output.txt
1	5 1 0 1 2 0	NO
2	5 1 3 2 5 4	YES

Листинг кода.

```
from measurements import *
from generator import *

@measure_performance
def heap_min_identifier(arr, n):
    for i in range((n-2)//2 + 1):
        l = 2*i + 1
        r = 2*i + 2

        if l < n and arr[i] > arr[l]:
            return 'NO'
        if r < n and arr[i] > arr[r]:
            return 'NO'

    return 'YES'

if __name__ == '__main__':
```

```

heap_generator(10**6)
with open('input.txt') as f:
    n = int(f.readline())
    items = list(map(int, f.readline().split()))

res_min = heap_min_identifer(items, n)
with open('output.txt', 'w') as f:
    f.write(res_min)
print(res_min)

```

Листинг кода для дополнительной функции

```

def heap_generator(n=None):
    if n is None:
        n = random.randint(1, 10**6)
        arr = random.choices(range(1, 10**6), k=n)

    with open('input.txt', 'w') as f:
        f.writelines([f'{n}\n', f'{" ".join(map(str, arr))}\n'])
    return None

```

Текстовое объяснение решения.

Алгоритм проверяет, является ли данный массив неубывающей пирамидой (min-heap), где каждый родитель не больше своих детей по индексам $2i+1$ и $2i+2$. Проверяются только внутренние узлы от 0 до $((n-2)//2)$, поскольку листья (индексы от $((n-2)//2)+1$ до $n-1$) не имеют детей и свойство кучи для них тривиально выполняется. Для каждого узла i вычисляются индексы левого ($2i+1$) и правого ($2i+2$) детей; если левый ребёнок существует ($l < n$) и $arr[i] > arr[l]$, или правый ребёнок существует ($r < n$) и $arr[i] > arr[r]$, то свойство нарушается и сразу возвращается 'NO'. Цикл останавливается при первом нарушении для оптимизации. Генератор создаёт тестовые данные случайного размера до 10^6 , записывая их в input.txt в требуемом формате. Сложность $O(n)$ достигается за счёт проверки каждого узла максимум один раз, что укладывается в ограничение 2 секунды даже для $n=10^6$. Выводится 'YES', если все проверенные пары родитель-ребёнок удовлетворяют условию $a_i \leq a_{2i+1}$ и $a_i \leq a_{2i+2}$.

Результат работы кода на примерах из задания:

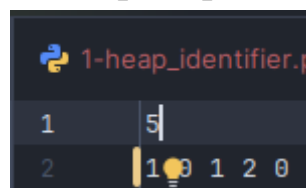


Рисунок 45 – Файл input.txt с входными данными из примера 1

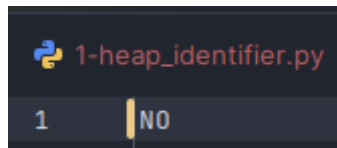


Рисунок 46 – Результат работы кода для данных из примера 1, записанный в output.txt

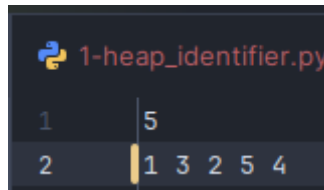


Рисунок 47 – Файл input.txt с входными данными из примера 2

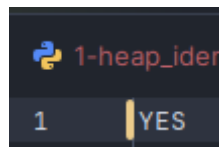


Рисунок 48 – Результат работы кода для данных из примера 2, записанный в output.txt

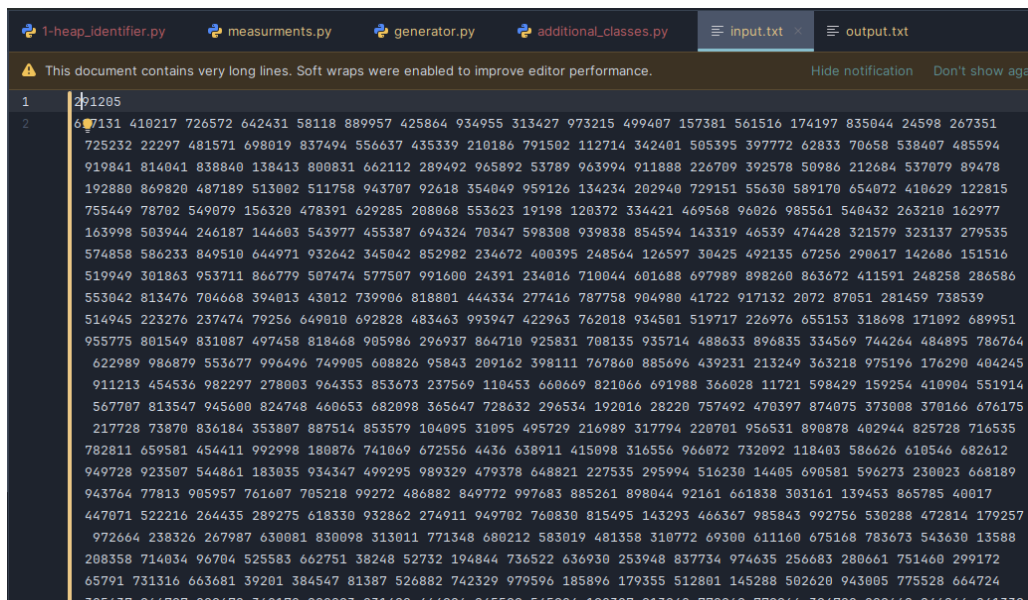


Рисунок 49 – Файл input.txt со сгенерированными входными данными (n=291205)

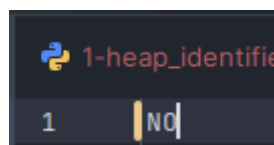


Рисунок 50 – Результат работы кода для сгенерированных данных, записанный в output.txt

Результат работы кода на максимальных и минимальных значениях:

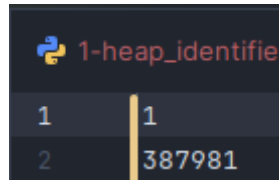


Рисунок 51 – Минимальное вводимое значение в input.txt

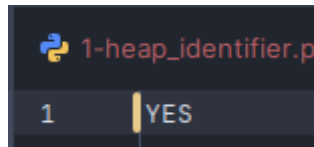


Рисунок 52 – Результат работы кода для значения $n = 1$, записанный в output.txt

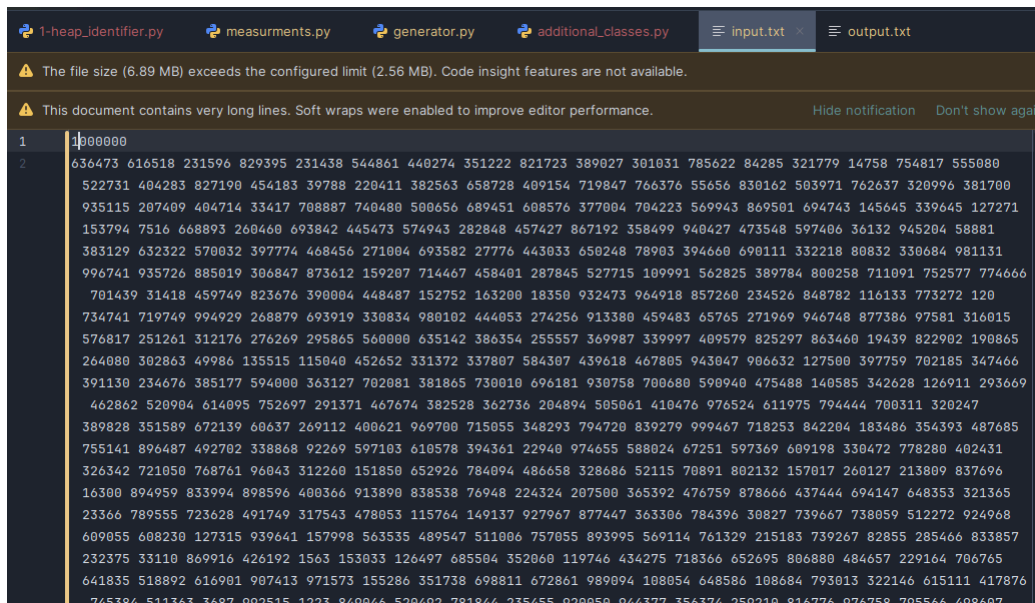


Рисунок 53 – Максимальное вводимое значение в input.txt

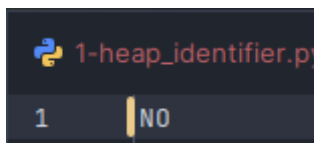


Рисунок 54 – Результат работы кода для значения $n = 1000000$, записанный в output.txt

Таблица затрат памяти и времени по заданию 1:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0000 сек	0.15 KB
Пример 1 из задачи	0.0000 сек	0.15 KB
Пример 2	0.0000 сек	0.15 KB

Сгенерированный пример	0.0000 сек	0.21 KB
Верхняя граница диапазона значений входных данных из текста задачи	0.0000 сек	0.21 KB

Вывод по задаче: Решение эффективно использует свойства массивного представления кучи и проверяет ровно те связи, которые требуются условием задачи.

Задача №5. Планировщик задач

В этой задаче вы создадите программу, которая параллельно обрабатывает список заданий. Во всех операционных системах, таких как Linux, MacOS или Windows, есть специальные программы, называемые планировщиками, которые делают именно это с программами на вашем компьютере.

У вас есть программа, которая распараллеливается и использует n независимых потоков для обработки заданного списка m заданий. Потоки берут задания в том порядке, в котором они указаны во входных данных. Если есть свободный поток, он немедленно берет следующее задание из списка. Если поток начал обработку задания, он не прерывается и не останавливается, пока не завершит

обработку задания. Если несколько потоков одновременно пытаются взять задания из списка, поток с меньшим индексом берет задание. Для каждого задания вы точно знаете, сколько времени потребуется любому потоку, чтобы обработать это задание, и это время одинаково для всех потоков.

Вам необходимо определить для каждого задания, какой поток будет его обрабатывать и когда он начнет обработку.

- **Формат ввода или входного файла (input.txt).** Первая строка содержит целые числа n и m ($1 \leq n \leq 10^5$, $1 \leq m \leq 10^5$). Вторая строка содержит m целых чисел t_i - время в секундах, которое требуется для выполнения i -ой задания любым потоком ($0 \leq t_i \leq 10^9$). Все эти значения даны в том порядке, в котором они подаются на выполнение. **Индексы потоков начинаются с 0.**
- **Формат выходного файла (output.txt).** Выведите в точности m строк, причем i -ая строка (начиная с 0) должна содержать два целочисленных значения: индекс потока, который выполняет i -ое задание, и время в секундах, когда этот поток начал выполнять задание.
- Ограничение по времени. 6 сек.
- Ограничение по памяти. 512 мб.
- Пример 1:

input.txt	output.txt
2 5	0 0
1 2 3 4 5	1 0
	0 1
	1 2
	0 4

1. Два потока пытаются одновременно взять задания из списка, поэтому поток с индексом 0 фактически берет первое задание и начинает работать над ним в момент 0.
2. Поток с индексом 1 берет второе задание и начинает работать над ним также в момент 0.
3. Через 1 секунду поток 0 завершает первое задание, берет третье задание из списка и сразу же начинает его выполнять в момент времени 1.
4. Через секунду поток 1 завершает второе задание, берет четвертое задание из списка и сразу же начинает его выполнять в момент времени 2.
5. Наконец, еще через 2 секунды поток 0 завершает третье задание, берет пятое задание из списка и сразу же начинает его выполнять в момент времени 4.

- Пример 2:

input.txt
4 20
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

output.txt
0 0
1 0
2 0
3 0
0 1
1 1
2 1
3 1
0 2
1 2
2 2
3 2
0 3
1 3
2 3
3 3
0 4
1 4
2 4
3 4

Задания берутся 4 потоками по 4 штуки за раз, обрабатываются за 1 секунду, а затем приходит следующий набор из 4 заданий. Это происходит 5 раз, начиная с моментов 0, 1, 2, 3 и 4. После этого обрабатываются все $5 \times 4 = 20$ заданий.

- Что делать? Подумайте о последовательности событий, когда один из потоков становится свободным (в самом начале и позже, после завершения некоторого задания). Как применить очередь с приоритетами, чтобы имитировать обработку этих заданий в нужном порядке? Не забудьте рассмотреть случай, когда одновременно освобождаются несколько потоков.

Листинг кода

```
from measurements import *
from generator import *
from additional_classes import *

@measure_performance
def flows(tasks_arr, fnum):
    if len(tasks_arr) <= fnum:
        return [(i, 0) for i in range(len(tasks_arr))]
    pq = PriorityQueue(fnum)
    fin_flows = [None] * len(tasks_arr)

    for i in range(len(tasks_arr)):
        t = tasks_arr[i]
        min_time, min_flow = pq.get_min()
        fin_flows[i] = (min_flow, min_time)
        pq.increase_min(min_time + t)

    return fin_flows

if __name__ == '__main__':
```

```

flows_generator(10, 10**5)
with open('input.txt') as f:
    fn, tn = tuple(map(int, f.readline().split()))
    tasks_times = list(map(int, f.readline().split()))

res = flows(tasks_times, fn)
with open('output.txt', 'w') as f:
    for r in res:
        f.write(f'{" ".join(map(str, r))}\n')

```

Листинг кода дополнительных функций

```

class PriorityQueue:
    def __init__(self, flows):
        self.heap = [(0, i) for i in range(flows)]
        self.build_min_heap()

    def build_min_heap(self):
        n = len(self.heap)
        for i in range(n // 2 - 1, -1, -1):
            self.min_heapify(i)

    def min_heapify(self, ind, n=None):
        arr = self.heap
        n = len(arr) if n is None else n
        l = 2 * ind + 1
        r = 2 * ind + 2

        m = ind
        if l < n and arr[ind] > arr[l]:
            m = l
        if r < n and arr[m] > arr[r]:
            m = r

        if m != ind:
            arr[ind], arr[m] = arr[m], arr[ind]
            self.min_heapify(m)

    def get_min(self):
        time, flow_num = self.heap[0]
        return time, flow_num

    def increase_min(self, time):
        flow = self.heap[0][1]
        self.heap[0] = (time, flow)
        self.min_heapify(0)

```

```

def flows_generator(n=None, m=None):
    if n is None:
        n = random.randint(1, 10**5)
    if m is None:
        m = random.randint(1, 10**5)

    times = random.choices(range(0, 10**9 + 1), k=m)

    with open('input.txt', 'w') as f:

```

```
f.write(f'{n} {m}\n')
f.write(" ".join(map(str, times)))
```

Текстовое объяснение решения

Решение моделирует планировщик заданий через min-кучу потоков, отслеживающую время их следующего освобождения. Изначально все n потоков свободны в момент 0 и формируют кортежи $(0, \text{индекс_потока})$, где кортеж автоматически обеспечивает лексикографический порядок: при равных временах поток с меньшим индексом всегда считается "меньшим" и извлекается первым. Для каждого из m заданий извлекается поток с минимальным временем освобождения из корня кучи, ему назначается текущее задание с моментом старта равным этому времени, после чего время освобождения обновляется на $\text{start_time} + t$ и просеивается вниз через $\text{min_heapify}(0)$, восстанавливая свойство кучи. Особенность в том, что извлечение не удаляет поток из кучи, а просто обновляет корневой элемент и перестраивает структуру, что эффективнее стандартного extract_min с заменой на последний элемент. Случай малого количества заданий ($m \leq n$) обрабатывается отдельно тривиально - каждому потоку по одному заданию в момент 0. Генератор создаёт тесты со случайными n , m до 10^5 и временами до 10^9 . Сложность $O(m \log n)$ достигается за счёт $\log n$ на каждое задание для просеивания, что укладывается в 6 секунд.

Результат работы кода на примерах из задания:

```
5-flows.py
1 2 5
2 1 2 3 4 5
```

Рисунок 55 – Файл input.txt с входными данными из примера 1

```

.py      🐍 1
1        0 0
2        1 0
3        0 1
4        1 2
5        0 4

```

Рисунок 56 – Результат работы кода для данных из примера 1

[illegible]

Рисунок 57 – Файл input.txt с входными данными из примера 2

py	1-h
2	1 0
3	2 0
4	3 0
5	0 1
6	1 1
7	2 1
8	3 1
9	0 2
10	1 2
11	2 2
12	3 2
13	0 3
14	1 3
15	2 3
16	3 3
17	0 4
18	1 4
19	2 4
20	3 4

Рисунок 58 – Файл output.txt с входными данными из примера 2

55805 81998

```

1 175446993 688386759 559033458 385095042 318120876 465435459 594366311 219854051 374845932 173117616 169519931 300500991
2 116131093 13730666 520076614 443754799 749003368 889766995 902782686 857968307 114653631 730141195 322026544
3 148253216 569890519 214114475 204795633 357158927 902145269 425007314 113511646 598983174 59240658 326894886
4 664102168 803711535 2665105389 226354402 896192557 61798440 157940928 40882662 589237272 29528089 615568414 458445720
5 799892828 706437656 40956431 235309170 928692320 581480959 643330876 385860951 383425813 761861129 7343935 829994180
6 26078565 668763798 440459563 593401491 469917216 376995733 538331635 512271063 657540837 433395845 44007389 846679508
7 121812195 277752924 202448590 683057746 508917312 734055836 313277599 797878084 497277774 11964512 815379441 325217133
8 611784059 184000196 922212724 996376043 471822091 495637779 573121431 499407221 710652215 755175198 854858592
9 610840330 251169797 725760903 339901273 717945431 38590016 264319588 714304375 206721104 752307288 582190850 266518455
10 259469317 680438896 261522422 959773920 465727235 31939149 723528942 487096913 904068407 311726618 846158211
11 648053041 588852526 345527064 80636915 601328687 838359805 894683627 773605141 712264742 340027545 846763992 633375862
12 602649199 861977807 320548985 14979668 570339666 575913218 945945363 70184552 783123556 169321260 909042581 313967909
13 216608711 91203198 363509771 144083230 186450065 591532566 454288056 436977685 725327029 749056422 475693172
14 7979646811 971267198 367367862 188089480 26964482 945079272 189355727 279937364 67288959 18960029 41139237 672130284
15 434235695 861275473 947090343 721452308 132059924 479110128 386829257 91773879 929638835 666659998 433920050 654028690
16 19149437 516124704 321741741 685278232 733711777 674668535 266396097 267271095 471998382 255598677 789065023
17 899776877 161505465 269233498 352881512 555804840 381878088 55269922 176806057 738262510 117091662 843853595 502974458
18 128925451 530723324 232887685 628638605 652354610 276684029 104761413 711876220 561000834 406304537 229679704
19 891675058 115970247 522399260 972497876 964008324 757883758 508108869 224418728 207273816 652958548 366120248
20 564184858 453241442 690061917 451271318 905739052 792213137 477347063 508479246 880804066 564291685 787251428
21 577096275 659315310 688619174 2702623712 167545908 509234744 66098517 68817888 383883469 353985307 369555718 786055286
22 61290891 360946724 549332495 746863893 837842474 725173212 138454654 944212664 389526414 818757710 231932218

```

Рисунок 59 – Файл input.txt со сгенерированными входными данными
(n=55805, m=81998)

```

py 1-heap_ide
1 0 0
2 1 0
3 2 0
4 3 0
5 4 0
6 5 0
7 6 0
8 7 0
9 8 0
10 9 0
11 10 0
12 11 0
13 12 0
14 13 0
15 14 0
16 15 0
17 16 0
18 17 0
19 18 0
20 19 0
21 20 0
22 21 0
23 22 0
24 23 0
25 24 0

```

Рисунок 60 – Файл output.txt с входными данными из примера 3 (сгенерированного)

Результат работы кода на максимальных и минимальных значениях:

```

5-flows.py
1 1 1
2 462446074

```

Рисунок 61 – Минимальное вводимое значение в input.txt

```

py 1-t
1 0 0

```

Рисунок 62 – Результат работы кода для значения $n, m = 1$, записанный в output.txt

Рисунок 63 – Максимальное вводимое значение в input.txt

```

1 0 0
2 1 0
3 2 0
4 3 0
5 4 0
6 5 0
7 6 0
8 7 0
9 8 0
10 9 0
11 10 0
12 11 0
13 12 0
14 13 0
15 14 0
16 15 0
17 16 0
18 17 0
19 18 0
20 19 0
21 20 0
22 21 0
23 22 0
24 23 0
25 24 0

```

Рисунок 64 – Результат работы кода для значения $n, m = 100000$, записанный в output.txt

Таблица затрат памяти и времени по заданию 5:

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.0000 сек	0.15 KB
Пример 1	0.0000 сек	0.52 KB

Пример 2	0.0000 сек	0.64 KB
Сгенерированный пример	1.1695 сек	12.66 MB
Верхняя граница диапазона значений входных данных из текста задачи	0.0405 сек	9.15 MB

Вывод по задаче: Решение использует свойства кортежей для разрешения конфликтов при равных временах и минимизирует операции с кучей, сохраняя все потоки в единой структуре без поиска позиций, что идеально подходит для задачи планировщика с жёсткими ограничениями.

Вывод

В лабораторной работе были рассмотрены такие структуры данных как куча и очередь с приоритетами, также был рассмотрен алгоритм создания min-кучи.